Machine Learning II

Aula 3 — Fundamentos de Deep Learning

Daniel Yukimura, Francisco Ganacim
IMPA

18 de março de 2026

Treinando Redes Neurais

Otimização vs Aprendizado

$$\min_{\theta \in \Theta} \mathbb{E}_{(x,y)\sim p_{\text{data}}} \left[\ell\left(f_\theta(x), y\right) \right] \quad \longleftarrow \quad \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^{n} \ell\left(f_\theta(x_i), y_i\right)$$

$$(x_i, y_i) \overset{\text{i.i.d.}}{\sim} p_{\text{data}}$$

- Minimização do Risco Empirico
- Em machine learning otimização ocorre de maneira indireta

Treinando Redes Neurais

Otimização vs Aprendizado

$$\nabla_\theta \, \mathbb{E}_{(x,y)\sim p_{\text{data}}} \left[\ell \left(f_\theta(x), y \right) \right] \quad \approx \quad \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \, \ell \left(f_\theta(x_i), y_i \right)$$

- O gradiente do risco empírico deveria aproximar o real, desde que $\left(x_i, y_i \right) \overset{\text{i.i.d.}}{\sim} p_{\text{data}}$
- Calcular o gradiente em todo o dataset pode ser custoso computacionalmente!

Otimização vs Aprendizado

Classificação: $\ell\left(f_\theta(x), y\right) = \mathbf{1}_{[\, f_\theta(x) \neq y \,]}$ — Perda 0-1 **não é diferenciável !**

$$\min_{\theta \in \Theta} \mathbb{P}_{(x,y)\sim p_{\text{data}}} \left[\, f_\theta(x) \neq y \,\right] \quad \longleftarrow \quad \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}_{[\, f_\theta(x_i) \neq y_i \,]}$$

Otimização vs Aprendizado

Classificação: $\ell\left(f_\theta(x), y\right) = \mathbf{1}_{[\, f_\theta(x) \neq y \,]}$ — Perda 0-1 **não é diferenciável !**

$$\min_{\theta \in \Theta} \mathbb{P}_{(x,y)\sim p_{\text{data}}} \left[\, f_\theta(x) \neq y \,\right] \quad \longleftarrow \quad \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^{n} -\log p_\theta\left(y_i \mid x_i\right)$$

- Log-verossimilhança negativa
- A otimização pode ser ainda mais indireta!

Otimização vs Aprendizado

Critério de parada:

- Em otimização tradicional:

$$R(\theta) \leq \varepsilon \quad \text{ou} \quad \nabla_\theta R(\theta) \approx 0$$

- Em machine learning:

$$\mathbb{E}_{(x,y)\sim\hat{p}_{\text{val}}}\left[\ell\left(f_\theta(x), y\right)\right] \leq \varepsilon$$

- **Early stopping**: Salva os pesos do modelo com menor erro de validação durante o treino

Gradiente Estocástico

Aproximar o gradiente usando todo o conjunto de dados é custoso!

$$\nabla_\theta \, \mathbb{E}_{(x,y)\sim p_{\text{data}}} \left[\ell \left(f_\theta(x), y \right) \right] \approx \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \, \ell \left(f_\theta(x_i), y_i \right)$$

Na prática aproximamos por um subconjunto pequeno dos dados, $|S| \ll n$

$$\nabla_\theta \, \mathbb{E}_{(x,y)\sim p_{\text{data}}} \left[\ell \left(f_\theta(x), y \right) \right] \approx \frac{1}{|S|} \sum_{i \in S} \nabla_\theta \, \ell(f(x_i, \theta), y_i)$$

Gradiente Estocástico

O ganho de aumentar o tamanho do batch não cresce linearmente!

$$\sqrt{\mathrm{Var}\left[\frac{1}{|S|}\sum_{i\in S}\nabla_\theta\,\ell(f(x_i,\theta),y_i)\right]} \sim \frac{1}{\sqrt{|S|}}$$

- Gradiente estocástico permite **computação eficiente**, sem sacrificar a qualidade da aproximação.
- $|S|$ pode ser escolhido de acordo com o hardware disponível (e.g. número de amostras que podem ser processadas em paralelo).

Algoritmos de Otimização

Gradiente descendente (GD):

$$\theta_{t+1} = \theta_t - \eta \nabla R_n(\theta_t) \qquad R_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \, \ell \left(f_\theta(x_i), y_i \right)$$

Algoritmos de Otimização

Gradiente descendente (GD)

```
for e in range(n_epochs):
    output = model(train_input)
    loss = criterion(output, train_target)
    model.zero_grad()
    loss.backward()
```

Num. de operações = $\mathcal{O}(|\mathscr{D}|)$

Algoritmos de Otimização

Gradiente descendente estocástico (SGD):

$$\theta_{t+1} = \theta_t - \eta \nabla \hat{R}_S(\theta_t) \qquad \hat{R}_S(\theta) = \frac{1}{|S|} \sum_{i \in S} \nabla_\theta \, \ell \left(f_\theta(x_i), y_i \right)$$

Algoritmos de Otimização

Gradiente descendente estocástico (SGD)

```
optimizer = torch.optim.SGD(model.parameters(), lr = eta)

for epoch in range(epochs):
    for x_batch, y_batch in loader:
        y_hat = model(x_batch)
        loss = criterion(y_hat, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Momento

SGD pode oscilar muito em regiões com alta curvatura, ou ser muito devagar em regiões de baixa curvatura

$$m_{t+1} = \beta m_t + (1 - \beta) \nabla_\theta \hat{L}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta m_{t+1}$$

Momento usa uma combinação da direção anterior com a atual, prevenindo mudanças bruscas e preferindo direções relevantes.

visualização

Algoritmos de Otimização

SGD com Momento

```
optimizer = torch.optim.SGD(model.parameters(), lr = eta, momentum=0.9)

for epoch in range(epochs):
    for x_batch, y_batch in loader:
        y_hat = model(x_batch)
        loss = criterion(y_hat, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

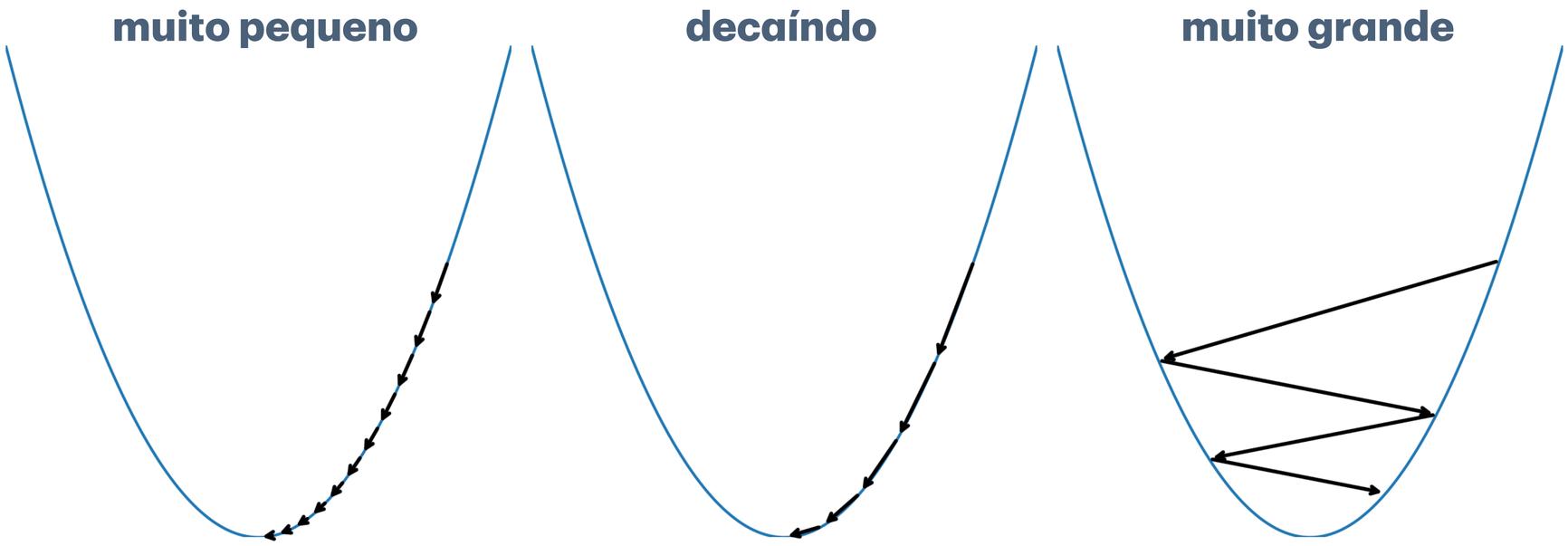
Taxa de aprendizado

O gradiente nos dá uma direção para minimizar a função, mas não a magnitude ideal

$$\theta_{t+1} = \theta_t - \eta_t \nabla \hat{L}(\theta_t)$$

Na prática, é necessário **diminuir ou adaptar a taxa de aprendizado** ao longo do treinamento, pois o erro das aproximações sucessivas durante o treinamento acaba acumulando.

Taxa de aprendizado



Taxa de aprendizado

Exemplo: Step Decay

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

Se $\eta_0 = 0.01$, $\gamma = 0.1$ e $s = 100$, então

- $t = 1 - 100 \rightarrow \eta_t = 0.01$
- $t = 101 - 200 \rightarrow \eta_t = 0.001$
- $t = 201 - 300 \rightarrow \eta_t = 0.0001$

Taxa de aprendizado

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=100, gamma=0.1)

for epoch in range(epochs):
    for x_batch, y_batch in loader:
        …
        [ LOOP padrão ]
        …
   scheduler.step()
```

Algoritmos de Otimização

Adam — Momentos Adaptativos: $g_t = \nabla \hat{L}\left(\theta_t\right)$

$$\boxed{m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t} \rightarrow \hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\boxed{v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2} \rightarrow \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

momento

escala adaptativa

Algoritmos de Otimização

Adam

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(epochs):
    for x_batch, y_batch in loader:
        …
        [ LOOP padrão ]
        …
```

Algoritmos de Otimização

`torch.optim`

```
optimizer = torch.optim.SGD( ... )
optimizer = torch.optim.Adam( ... )
optimizer = torch.optim.Adagrad( ... )
optimizer = torch.optim.RMSProp( ... )
optimizer = torch.optim.LBFGS( ... )
...
```

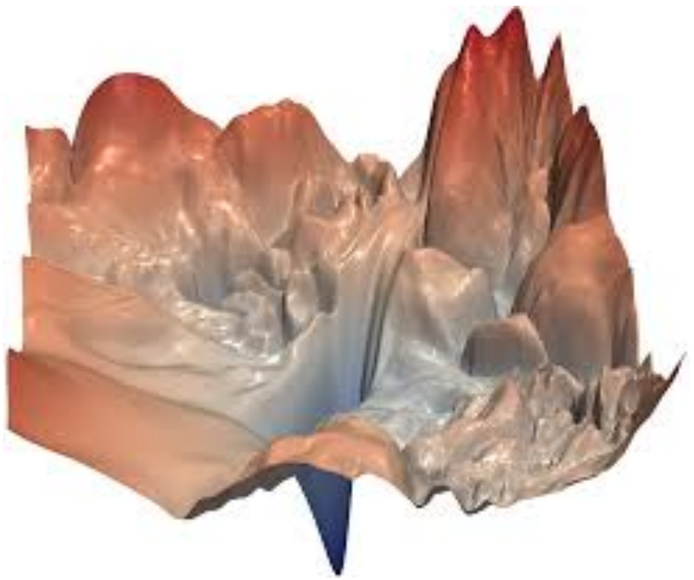
Treinando Redes Neurais

A geometria da função de perda

Otimização em redes neurais:

- Problema altamente **não convexo**
- **Alta dimensão**
- Não há garantias clássicas de convergência global

$$\min_{\theta \in \Theta} R_n(\theta)$$



Li et al. (2018) — Visualização da loss

Treinando Redes Neurais

A geometria da função de perda

Geometria em alta dimensão:

- Em alta dimensão temos muitos **pontos de sela** — Dauphin et al. (2014)
- Não linearidade tendem a gerar **mínimos locais ruins** — Yun et al. (2019)
- A **dimensão intrínseca dos dados** afeta a geometria da perda — Venturi et al. (2018)

Treinando Redes Neurais

A geometria da função de perda

Porque conseguimos treinar redes neurais na prática?

- **Overparametrization** parece tornar a geometria do problema mais facil de lidar
- O “ruído” gerado pelo SGD pode ajudar a **escapar pontos de sela** — Daneshmand et al. (2018)
- A estrutura do dado afeta a capacidade de generalização, criando um **viés implícito** no treinamento

Regularização

Generalização:

$$\min_{\theta \in \Theta} R(\theta) \quad \leftarrow \quad \min_{\theta \in \Theta} R_n(\theta) \quad \leftarrow \quad \min_{\theta \in \Theta} \hat{L}_{\text{aux}}(\theta)$$

- Como modificar o algoritmo de aprendizado de forma a melhorar generalização?

Regularização

A maneira mais comum de **regularizar** um problema de aprendizado é tentar **restringir o espaço de busca de parâmetros** adicionando uma penalização na perda

$$\min_{\theta \in \Theta} R_n(\theta) + \lambda \Omega(\theta)$$

- λ é um hiperparametro controla a ênfase dada a regularização.

Regularização

Regularização L^2:

$$\min_{\theta \in \Theta} R_n(\theta) + \lambda \|\theta\|_2$$

- Tende a melhorar o condicionamento da Hessiana da perda, reduzindo instabilidades e sensibilidade a ruído.

Regularização

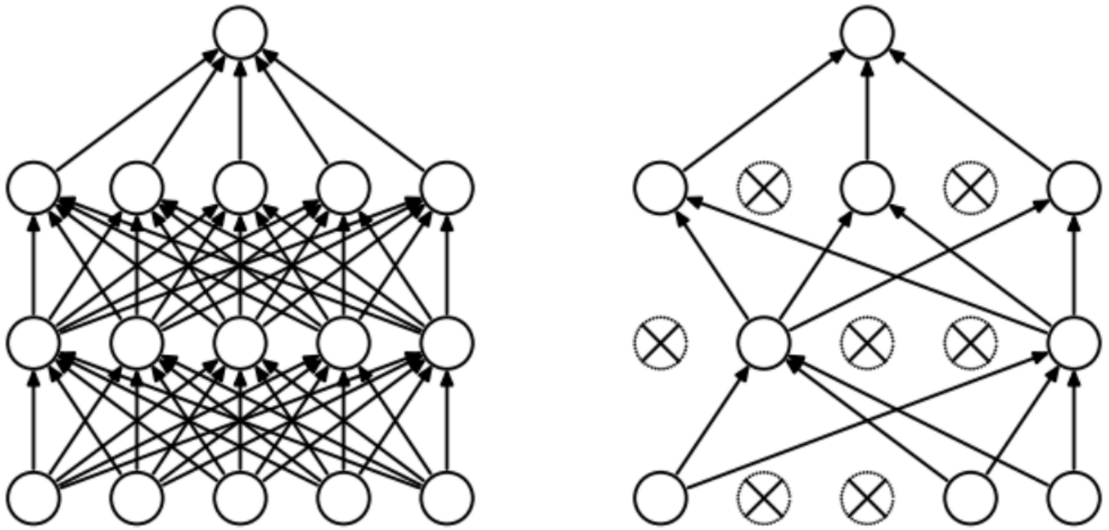
Regularização L^1:

$$\min_{\theta \in \Theta} R_n(\theta) + \lambda \|\theta\|_1$$

- Nesse caso, controlamos a capacidade do modelo ao promover esparsidade.
- O método **Lasso** é uma forma de regularização L^1 muito utilizada em problemas de alta dimensão.

Dropout

- O dropout pode ser visto como uma aproximação eficiente de ensemble learning.
- Cada iteração treina uma sub-rede diferente, obtida ao remover neurônios aleatoriamente.



(a) Standard Neural Net (b) After applying dropout.

Dropout

- A cada iteração, uma sub-rede é escolhida aleatoriamente, resultando em um ensemble com dependência estatística.
- Idealmente, a predição corresponderia à média ponderada sobre todas as sub-redes:

$$\sum_{\mathcal{S} \subseteq \mathcal{N}} p(\mathcal{S}) p_{\mathcal{S}}(y \mid x)$$

- No entanto, essa soma possui um número exponencial de termos, tornando-se intratável na prática.
- Em vez disso, utiliza-se uma aproximação simples: reescalar os pesos pela probabilidade de ativação dos neurônios na fase de teste.

Dropout

```
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(256, 10)
)
```

Dropout

```
model.train()  # dropout ON

for epoch in range(epochs):
   for x_batch, y_batch in train_loader:
        y_hat = model(x_batch)
        loss = criterion(y_hat, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    model.eval()   # dropout OFF
    val_loss, val_acc = evaluate(model, val_loader)
```

Desafios no treinamento de Redes Neurais

Vanishing / Exploding Gradients

Gradientes em redes profundas:

- Gradientes são produtos sucessivos (regra da cadeia)

$$\frac{\partial \mathscr{L}}{\partial \mathbf{h}^{(0)}} \approx \prod_{\ell=1}^{L} \left(\mathbf{W}^{(\ell)} \cdot \sigma'(\mathbf{z}^{(\ell)}) \right)$$

Consequências:

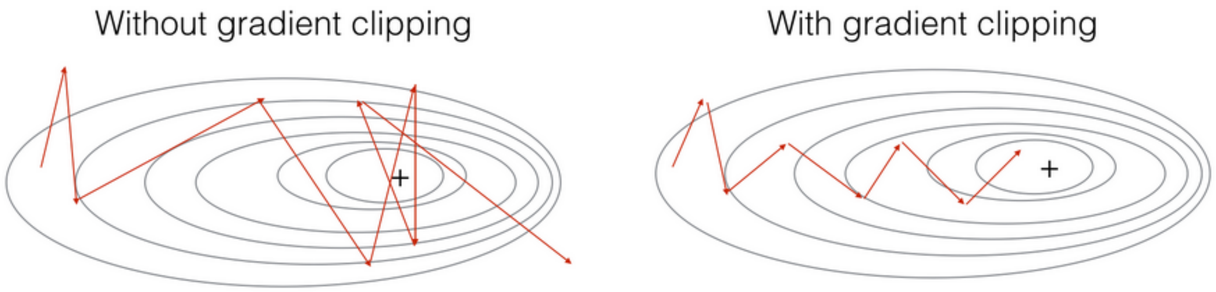
- Camadas iniciais aprendem lentamente
- Treinamento instável
- Sensível à inicialização

Desafios no treinamento de Redes Neurais

Vanishing / Exploding Gradients

Soluções:

- ReLU / variantes
- Inicialização (He, Xavier)
- ResNet (skip connections)
- Gradient clipping



$$g \leftarrow \frac{g}{\|g\|} \cdot \min\left(\|g\|, c\right)$$

Desafios no treinamento de Redes Neurais

Gradient Clipping

```
for x, y in dataloader:
    optimizer.zero_grad()
    y_hat = model(x)
    loss = criterion(y_hat, y)
    loss.backward()

    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

    optimizer.step()
```

Inicialização

Onde inicializamos a rede tem muita influência no treinamento. Algumas regiões do espaço tem melhores condições de atingir um bom mínimo

Uma forma de inicializar é tentar preservar escalas entre as camadas

$$\mathrm{Var}(h^{(\ell)}) \approx \mathrm{Var}(h^{(\ell-1)})$$

Exemplos:

- Xavier (Glorot) — ativações simétricas $$W_{ij}^{(\ell)} \sim \mathcal{U}\left(-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}\right)$$
- He — ReLU $$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0, \frac{2}{n}\right)$$

Batch Normalization

Normalização das pré-ativações

- Em cada camada, normalizamos as pré-ativações no mini-batch
- Controlamos a escala antes da não-linearidade $z_i^{(\ell)} = W^{(\ell)} h_i^{(\ell-1)}$

$$\mu_B^{(\ell)} = \frac{1}{m} \sum_{i=1}^{m} z_i^{(\ell)}, \quad (\sigma_B^{(\ell)})^2 = \frac{1}{m} \sum_{i=1}^{m} (z_i^{(\ell)} - \mu_B^{(\ell)})^2$$

$$\hat{z}_i^{(\ell)} = \frac{z_i^{(\ell)} - \mu_B^{(\ell)}}{\sqrt{(\sigma_B^{(\ell)})^2 + \epsilon}}, \qquad h_i^{(\ell)} = \sigma\left(\gamma^{(\ell)} \hat{z}_i^{(\ell)} + \beta^{(\ell)}\right)$$

Dados e Generalização

Data augmentation:

$$x \to T(x), \quad T \in \mathcal{T}$$

- Define invariâncias desejadas
- Induz uma nova distribuição para os dados

$$(x, y) \sim p_{\text{data}} \Rightarrow (T(x), y) \sim p_T$$

Resumo da aula:

Treinar redes neurais envolve:

- **Otimização**
  - GD, SGD, Adam
  - Geometria não convexa
- **Estabilidade**
  - BatchNorm, clipping, inicialização
- **Regularização**
  - Dropout, early stopping
- **Dados**
  - Distribuição, generalização